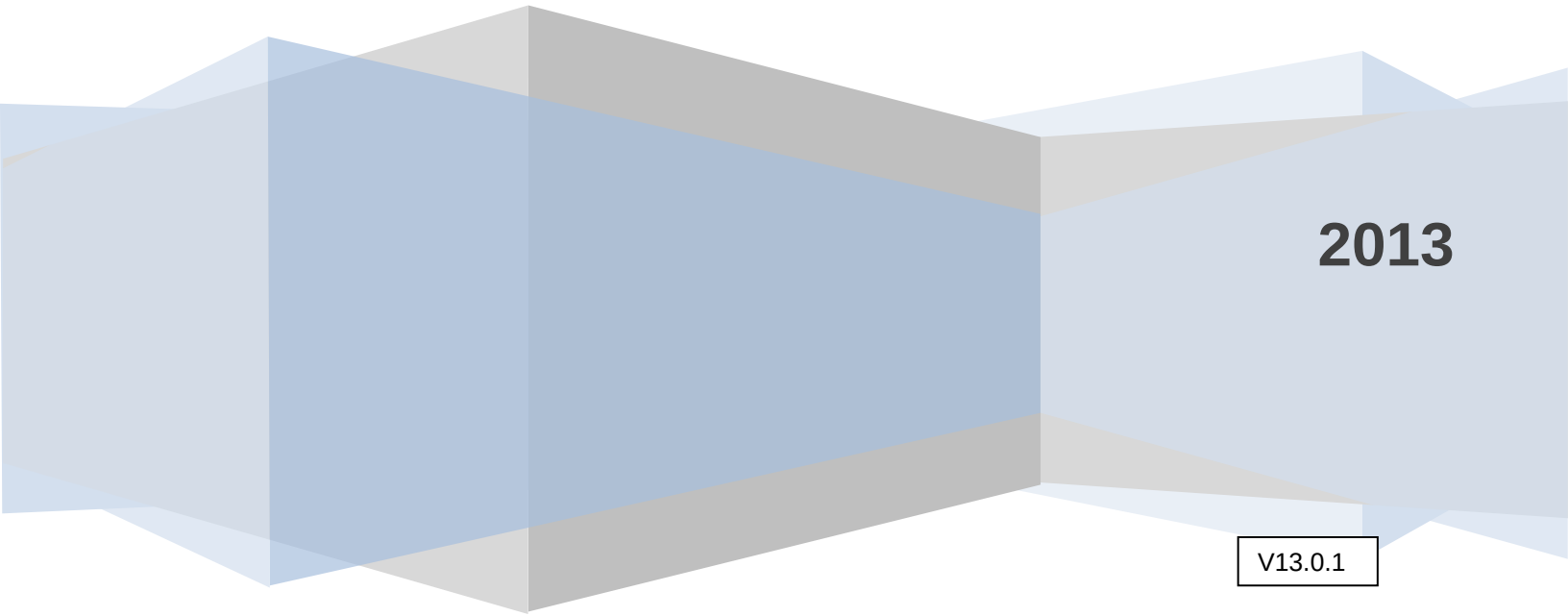
Texas Instruments, Inc.
C2000 Systems and Applications

Digital Motor Control

Software Library:
Special Math Blocks



2013

V13.0.1

Contents

Introduction

The digital motor control library is composed of C functions (or macros) developed for C2000 motor control users. These modules are represented as modular blocks in C2000 literature in order to explain system-level block diagrams clearly by means of software modularity. The DMC library modules cover nearly all of the target-independent mathematical macros and target-specific peripheral configuration macros, which are essential for motor control.

In the DMC library, each module is separately documented with source code, use, and background technical theory. The DMC library components have been used by TI to provide system-level motor control examples. In the motor control code, all DMC library modules are initialized according to the system specific parameters, and the modules are inter-connected to each other. At run-time the modules are called in order. Each motor control system is built using an incremental build approach, which allows some sections of the code to be built at a time, so that the developer can verify each section of the application one step at a time. This is critical in real-time control applications, where so many different variables can affect the system, and where many different motor parameters need to be tuned.

All DMC modules allow users to quickly build, or customize their own systems. The library supports three principal motor types (induction motor, BLDC and PM motors) but is not limited to these motors.

All these modules are provided in the downloadable version of ControlSuite. In addition to these, few additional modules are developed by TI related to the control PMSM motors, for superior performance. The original SMO for sensorless control of PMSM has some scope for improvements and these are covered by enhanced SMO module (eSMO) that provides the following features

1. SMO filter angle compensation
2. Bidirectional speed control
3. Inverter dead band compensation

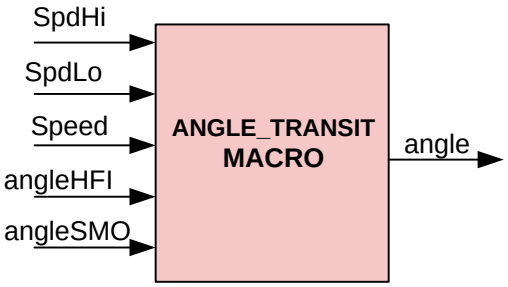
eSMO still cannot handle / estimate speeds close to zero as the bemf is too small. However, for an interior PM motor that has rotor saliency, an alternate position sensing approach can be used based on high frequency injection at zero and low speeds.

ANGLE_TRANSIT

Transition between HFI and SMO based Rotor Angle Estimation

Description

This module handles transitioning the control between HFI and SMO based rotor angle estimations.



Availability

C interface version

Module Properties

Type: Target Independent

Target Devices: 28x Fixed or Floating Point

C Version File Names: transition.c

IQmath library files for C: IQmathLib.h, IQmath.lib

C Interface

Object Definition

The structure of TRANSITION object is defined by following structure definition

```
typedef struct {
        _iq   spd;          // Input: Motor speed (Q24)
        _iq   spdLo;        // Input: Motor speed below which HFI angle estimate is used (Q24)
        _iq   spdHi;        // Input: Motor speed above which SMO angle estimate is used (Q24)
        _iq   angleHFI;     // Input: Angle estimate using HFI (Q24)
        _iq   angleSMO;     // Input: Angle estimate using SMO (Q24)
        _iq   angle;        // Output: Estimated Angle (Q24)
        _iq   scale;        // Parameter: Scale to compute weighted gain Ksmo (Q24)
    } TRANSITION;           // Data type created
```

Item	Name	Description	Format*	Range(Hex)
Inputs	spdHi	Rotor Speed	GLOBAL_Q	80000000-7FFFFFFF
	spdLo	Rotor Speed	GLOBAL_Q	80000000-7FFFFFFF
	spd	Rotor Speed	GLOBAL_Q	80000000-7FFFFFFF
	angleHFI	Raw resolver angle	GLOBAL_Q	80000000-7FFFFFFF (0 – 360 degree)
	angleHFI	Raw resolver angle	GLOBAL_Q	80000000-7FFFFFFF (0 – 360 degree)
Outputs	angle	Rotor angle in per-unit	GLOBAL_Q	80000000-7FFFFFFF
TRANSITION parameters	scale	A constant to compute Ksmo	GLOBAL_Q	0-7FFFFFFF

*GLOBAL_Q valued between 1 and 30 is defined in the IQmathLib.h header file.

Special Constants and Data types

TRANSITION

The module definition is created as a data type. This makes it convenient to instance a transition interface for angle estimation. To create multiple instances of the module, simply declare variables of type TRANSITION.

TRANSITION_DEFAULTS

Structure symbolic constant to initialize TRANSITION module. This provides the initial values to the terminal variables as well as method pointers.

Module Usage

Instantiation

The following example instances two TRANSITION objects

```
TRANSITION transition1, transition2;
```

Initialization

To Instance pre-initialized objects

```
TRANSITION transition1 = TRANSITION _DEFAULTS;
TRANSITION transition2 = TRANSITION _DEFAULTS;
```

Invoking the computation macro

```
ANGLE_TRANSIT (&transition1);
ANGLE_TRANSIT (&transition2);
```

Example

The following pseudo code provides the information about the module usage.

```
main()
{


}


void interrupt periodic_interrupt_isr()
{
       transition1.spd      = speed1;          // Pass inputs to transition1
       transition1.angleHFI = hfiAngle1;       // Pass inputs to transition1
       transition1.angleSMO = smoAngle1;       // Pass inputs to transition1

       transition2.spd      = speed2;          // Pass inputs to transition2
       transition2.angleHFI = hfiAngle2;       // Pass inputs to transition2
       transition2.angleSMO = smoAngle2;       // Pass inputs to transition2

       ANGLE_TRANSIT (&transition1);    // Call compute macro for transition1
       ANGLE_TRANSIT (&transition2);    // Call compute macro for transition2

       rotor1pos   = transition1.angle;        // Access output of transition1
       rotor2pos   = transition2.angle;        // Access output of transition2
}
```

Technical Background

When the rotor speed is below a set limit ‘spdLo’, angleHFI is used as the position of rotor. When the rotor speed is above a set limit ‘spdHi’, angleSMO is used as the position of rotor. However, if the rotor speed is in between these limits, the control is said to be in transition, where the current speed of the rotor is compared against these two and some weighted median value is derived. The closer the speed is to spdLo, angleHFI will contribute more to the median value while angleSMO will contribute less. Likewise, their roles reverse if the speed is closer to spdHi, and is mathematically closer to what is given below

Angle_out = (angleSMO * (spd - spdLo) + angleHFI * (spdHi - spd)) / (spdHi – spdLo)

Ksmo is calculated as follows

Ksmo = (spd - spdLo) * scale / (spdHi – spdLo)

where,

‘scale’ is used to trim the value of Ksmo to ensure reliable transition, defaulting to 1.0

Transition angle estimate can be revised as

Angle_out = angleSMO * Ksmo + angleHFI * (1 - Ksmo)

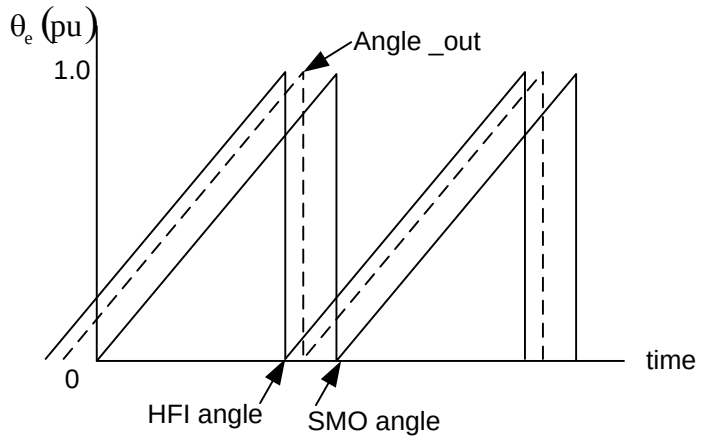
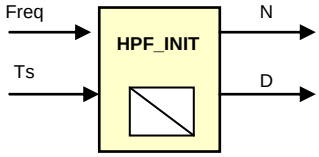


Figure 2: Angle calculation when speed is within transition limits

HPF_INIT — *Initialize High Pass Filter Coefficients*

Description

This module sets up coefficients N and D for high pass filters. The same coefficients can be used by multiple high pass filter modules



Availability

C interface version

Module Properties

Type: Target Independent

Target Devices: 28x Fixed or Floating Point

C Version File Names: HFI.c

IQmath library files for C: IQmathLib.h, IQmath.lib

C Interface

Object Definition

The structure of HPF_COEFF object is defined by following structure definition

```
typedef struct {  _iq  freq;        // Input: filter corner frequency
                  _iq  PiT;         // Input: constant = PI*Ts
                  _iq  wTby2;       // variable
                  _iq  N;           // Output: coefficient N
                  _iq  D;           // Output: coefficient D
                } HPF_COEFF;
```

Item	Name	Description	Format*	Range(Hex)
Inputs	freq	Filter corner frequency	GLOBAL_Q	80000000-7FFFFFFF
	PiT	Constant : Pi.Ts	GLOBAL_Q	80000000-7FFFFFFF
Variable	wTby2	Intermediate variable	GLOBAL_Q	80000000-7FFFFFFF
Outputs	N	Filter coefficient	GLOBAL_Q	80000000-7FFFFFFF
	D	Filter coefficient	GLOBAL_Q	80000000-7FFFFFFF

*GLOBAL_Q valued between 1 and 30 is defined in the IQmathLib.h header file.

Special Constants and Data types

HPF_COEFF

The module definition is created as a data type. This makes it convenient to instance an interface to setting up the coefficients of high pass filter. To create multiple instances of the module simply declare variables of type HPF_COEFF.

HPF_COEFF_DEFAULTS

Structure symbolic constant to initialize HPF_COEFF module. This provides the initial values to the terminal variables as well as method pointers.

Module Usage

Instantiation
The following example instances two HPF_COEFF objects
```
HPF_COEFF  hpf_coeff1,hpf_coeff2;
```

Initialization
To Instance pre-initialized objects
```
HPF_COEFF hpf_coeff1 = HPF_COEFF _DEFAULTS;
HPF_COEFF hpf_coeff2 = HPF_COEFF _DEFAULTS;
```

Invoking the computation macro
```
HPF_INIT(&hpf_coeff1);
HPF_INIT(&hpf_coeff2);
```

Example
The following pseudo code provides the information about the module usage.

```
main()
{
       hpf_coeff1.freq = freq1;         // Pass inputs to hpf_coeff1
       hpf_coeff1.PiT  = _IQ(PI*T);     // Pass inputs to hpf_coeff1

       hpf_coeff2.freq = freq2;         // Pass inputs to hpf_coeff2
       hpf_coeff2.PiT  = _IQ(PI*T);     // Pass inputs to hpf_coeff2

       HPF_INIT(&hpf_coeff1);           // Call init function for hfp_coeff1
       HPF_INIT(&hpf_coeff2);           // Call init function for hpf_coeff2
}

void interrupt periodic_interrupt_isr()
{

}
```

Technical Background

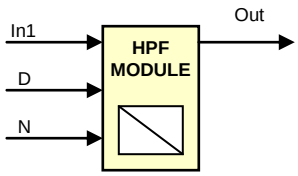
Calculates the coefficients for high pass filter (HPF), where the HPF is given by:

Out = N*(In1-In0) + D*Out
In0 = In1

The coefficients N and D are calculated using bilinear transformation as follows:

$N = 2 / (2 + \omega_0 T_s)$
$D = (2 - \omega_0 T_s) / (2 + \omega_0 T_s)$,
where,
$\omega_0 = 2*pi*F_0$, the corner frequency of filter

Description This module performs high pass filtering of the inputs based on coefficients D and N.



Availability C interface version

Module Properties **Type:** Target Independent

Target Devices: 28x Fixed or Floating Point

C Version File Names: HFI.c

IQmath library files for C: IQmathLib.h, IQmath.lib

C Interface

This module uses another object HPF_COEFF and a new object HPF as defined below. HPF_COEFF object contains parameters needed by HPF_MODULE. Refer to the section on HPF_COEFF for details.

Object Definition

The structure of HPF object is defined by following structure definition

```
typedef struct {  _iq  In1;          // Input: present input
                  _iq  In0;          // Input: previous input
                  _iq  Out;          // Output:
               } HPF;
```

Item	Name	Description	Format*	Range(Hex)
Inputs	In1	Latest input	GLOBAL_Q	80000000-7FFFFFFF
	In0	Previous input	GLOBAL_Q	80000000-7FFFFFFF
Outputs	Out	Output	GLOBAL_Q	80000000-7FFFFFFF

*GLOBAL_Q valued between 1 and 30 is defined in the IQmathLib.h header file.

Special Constants and Data types

HPF

The module definition is created as a data type. This makes it convenient to instance an interface to the high pass filter. To create multiple instances of the module simply declare variables of type HPF.

HPF_DEFAULTS

Structure symbolic constant to initialize HPF module. This provides the initial values to the terminal variables as well as method pointers.

Module Usage

Instantiation

The following example instances two HPF objects and a HPF_COEFF object

```
HPF  hpf1,hpf2;
HPF_COEFF hpf_coeff1;
```

Initialization

To Instance pre-initialized objects

```
HPF hpf1 = HPF_DEFAULTS;
HPF hpf2 = HPF_DEFAULTS;

HPF_COEFF hpf_coeff1 = HPF_COEFF_DEFAULTS;
```

Invoking the computation macro

```
HPF_MODULE (&hpf1, &hpf_coeff1);
HPF_MODULE (&hpf2, &hpf_coeff1);
```

Example

The following pseudo code provides the information about the module usage.

```
main()
{
       hpf_coeff1.freq = _IQ(18.0);   // in Hz
       hpf_coeff1.PiT  = _IQ(PI*T);
       HPF_INIT(&hpf_coeff1);
}

void interrupt periodic_interrupt_isr()
{
       hpf1.In1 = ds1;                  // Pass inputs to hpf1
       hpf2.In1 = ds2;                  // Pass inputs to hpf2

       HPF_ MODULE(&hpf1, &hpf_coeff1); // Call compute function for hpf1
       HPF_ MODULE(&hpf2, &hpf_coeff1); // Call compute function for hpf2

       de1 = hpf1.Out;                  // Access the outputs of hpf1
       de2 = hpf2.Out;                  // Access the outputs of hpf2
}
```

Technical Background

Implements the following equations:

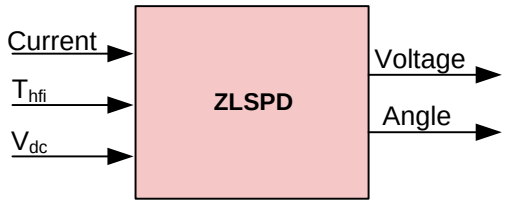
Out = N*(In1-In0) + D*Out
In0 = In1

The corner frequency of the filter is set by coefficients N and D. Alternatively, knowing corner frequency ω_0, N and D can be calculated using bilinear transformation as follows:

$$N = 2 / (2 + \omega_0 T_s)$$
$$D = (2 - \omega_0 T_s) / (2 + \omega_0 T_s)$$

Description

This module handles injecting high frequency signals in to the stator of a salient pole PMSM and identifies rotor position. It uses a few different variable structures to cover certain sections of this implementation



Availability

C interface version

Module Properties

Type: Target Independent

Target Devices: 28x Fixed or Floating Point

C Version File Names: hfi.c

IQmath library files for C: IQmathLib.h, IQmath.lib

C Interface

Objects Definition

This module uses a couple of objects, HFI and NS_ID

HFI object

The structure of HFI object is defined by following structure definition

```
typedef struct {
        _iq  Kp_IPD,        // Kp during IPD
        _iq  Kp_RUN,        // Kp during RUN
        _iq  base_wTs,      // base_wTs = BASE_FREQ * T ,for lib
        _iq  dutyMax,       // max duty cycle
        _iq  volt_run,      // minimum regulation Vdc during RUN
        _iq  volt_ipd,      // minimum regulation Vdc during IPD

        _iq  speedEst,      // estimated speed
        _iq  thetaEst,      // estimated angle
        _iq  duty;          // operating duty cycle

        Uint16   Squ_PRD_set,   // square wave period
        Uint16   HFI_Time1,     // end of initial alignment completion
        Uint16   HFI_Time2,     // end of post alignment no current (PAUSE) time

        Uint16   HFI_Status;    // state machine state
    }  HFI;
```

Item	Name	Description	Format*	Range(Hex)
Inputs	Kp_IPD	Prop gain for loop PLL @ IPD	GLOBAL_Q	80000000-7FFFFFFF
	Kp_RUN	Prop gain for loop PLL @ RUN	GLOBAL_Q	80000000-7FFFFFFF
	base_wTs	Base angle increment per Ts	GLOBAL_Q	80000000-7FFFFFFF
	dutyMax	Max duty cycle in HFI	GLOBAL_Q	80000000-7FFFFFFF
	Volt_run	HFI voltage during run	GLOBAL_Q	80000000-7FFFFFFF
	Volt_ipd	HFI voltage during IPD	GLOBAL_Q	80000000-7FFFFFFF
	Squ_PRD_set	HFI half period reference	Q0	0-FFFF
	HFI_Time1	Initial alignment time	Q0	0-FFFF
	HFI_Time2	End of post alignment pause	Q0	0-FFFF
	HFI_Status	State Machine - state	Q0	0-FFFF
Outputs	speedEst	Estimated speed	GLOBAL_Q	80000000-7FFFFFFF
	thetaEst	Angle estimate	GLOBAL_Q	80000000-7FFFFFFF
	Duty	Operating duty cycle	GLOBAL_Q	0-7FFFFFFF

*GLOBAL_Q valued between 1 and 30 is defined in the IQmathLib.h header file.

NS_ID object

The structure of NS_ID object is defined by following structure definition

```
typedef struct {
        Uint16   cntON;          // number of ON pulses within cntPRD below
        Uint16   cntPRD;         // pulse period per switching state
        Uint16   PWM_PeriodMax;  // PWM timer PRD
        Uint16   PWM_ch[3];      // ePWM channels to be used by lib
} NS_ID;
```

Item	Name	Description	Format*	Range(Hex)
Inputs	cntPRD	Pulse period per switching state	Q0	0-FFFF
	cntON	Number of ON pulses within cntPRD	Q0	0-FFFF
	PWM_PeriodMax	ePWM timer PRD	Q0	0-FFFF
	PWM_ch[3]	Array of ePWM channels to use by library	Q0	0-FFFF

*GLOBAL_Q valued between 1 and 30 is defined in the IQmathLib.h header file.

Special Constants and Data types

HFI
The module definition is created as a data type. This makes it convenient to instance a high frequency injection control module. To create multiple instances of the module simply declare variables of type HFI.

HFI_DEFAULTS
Structure symbolic constant to initialize HFI module. This provides the initial values to the terminal variables as well as method pointers.

NS_ID
The module definition is created as a data type. This makes it convenient to instance a high frequency injection control module. To create multiple instances of the module simply declare variables of type NS_ID.

NS_ID_DEFAULTS
Structure symbolic constant to initialize NS_ID module. This provides the initial values to the terminal variables as well as method pointers.

ZLSPD module

Unlike other modules in math library, this module is architected differently. It takes in pointers to various variables as inputs and processes the data. The structure of ZLSPD module is defined as follows

void ZLSPD(HPF *iq, HPF_COEFF *k, HFI *h, NS_ID *n, CLARKE *I, PHASEVOLTAGE *v)

Module Usage

Instantiation

The following example instances two HFI and NS_ID objects

```
HFI hfi1, hfi2;
NS_ID ns_id1, ns_id2;
```

Initialization

To Instance pre-initialized objects

```
HFI hfi1 = HFI _DEFAULTS;
HFI hfi2 = HFI _DEFAULTS;

NS_ID  ns_id1 = NS_ID_DEFAULTS;
NS_ID  ns_id2 = NS_ID_DEFAULTS;

HPF iq1, iq2;
HPF_COEFF hpf_coeff1, hpf_coeff2;
CLARKE i1, i2;
PHASEVOLTAGE v;
```

Invoking the computation macro

```
ZLSPD (&iq1, & hpf_coeff1, &hfi1, &ns_id1, &i1, &v);
ZLSPD (&iq2, & hpf_coeff2, &hfi2, &ns_id2, &i2, &v);
```

Example

The following pseudo code provides the information about the module usage.

```
main()
{
        // Set up hpf_coeff1, hpf_coeff2
        ……..
        // Set up hfi1, hfi2
        ……..
        // Set up ns_id1, ns_id2
        ……..
}

void interrupt periodic_interrupt_isr()
{
       hpf _Iq1.In1 = currentPark1.Qs;  // Pass inputs to Hpf_Iq
       hfi1.DcBusVolt      = Vbus;      // Pass inputs to hfi1

       hpf_Iq2.In1  = currentPark2.Qs;  // Pass inputs to Hpf_Iq
       hfi2.DcBusVolt      = Vbus;      // Pass inputs to hfi1

       // Call compute functions for hfi1 and hfi2
       ZLSPD (&hpf_id1, &hpf_iq1, &hpf_coeff1, &hfi1, &volt1);
       ZLSPD (&hpf_id2, &hpf_iq2, &hpf_coeff2, &hfi2, &volt2);

       rotor1pos   = hfi1.thetaEst;           // Access output of hfi1
       rotor2pos   = hfi2.thetaEst;           // Access output of hfi2
}
```

Technical Background

High frequency injection based rotor position estimation is applicable only for PMSM motors with saliency. Fig. 1 shows the block diagram of this estimation scheme. There are two types of salient motors, surface mount PM motors with saturation induced saliencies on the teeth on their rotors, and interior PM motors with magnets buried in rotors. Due to saliency, the inductances of the motor along d and q axes vary as a function of rotor position, which will impact the currents flowing through stator windings. In this case, the motor currents help to, not only control the torque, but also to estimate the position of rotor.

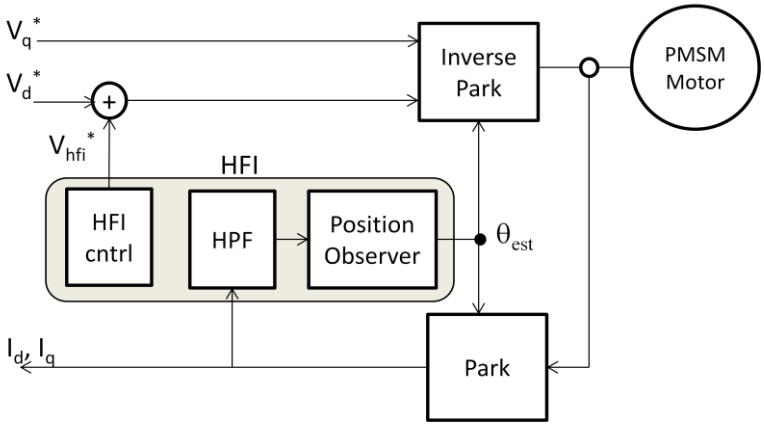


Fig. 1 HFI based rotor position estimation

The spatial variation of magnetic flux and stator inductance in an interior PM motor, as a function of rotor position, are shown in fig. 2(a) and 2(b) respectively. Since back emf (BEMF) is a function of magnetic flux and rotor speed, it can be used to back track the rotor position when there is sufficient bemf to work with, such as during mid to high speeds. But, at zero and very low speeds, the back emf is too small to give reliable results. However, as shown in (b), the inductance variation due to motor saliency is dependant only on shaft position and not on speed. Therefore tracking the inductance will help to estimate the position of rotor at zero and low speeds. This is a two step process and will be explained subsequently.

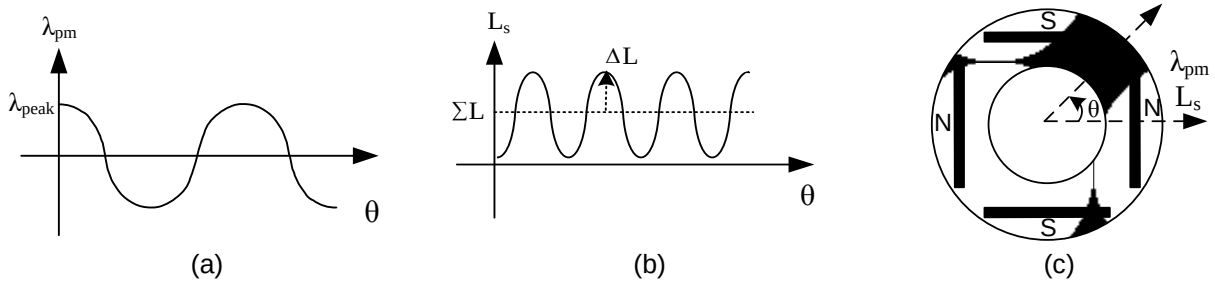


Fig. 2 Spatial signals of (a) magnet flux and (b) inductance in (c) interior PM motor

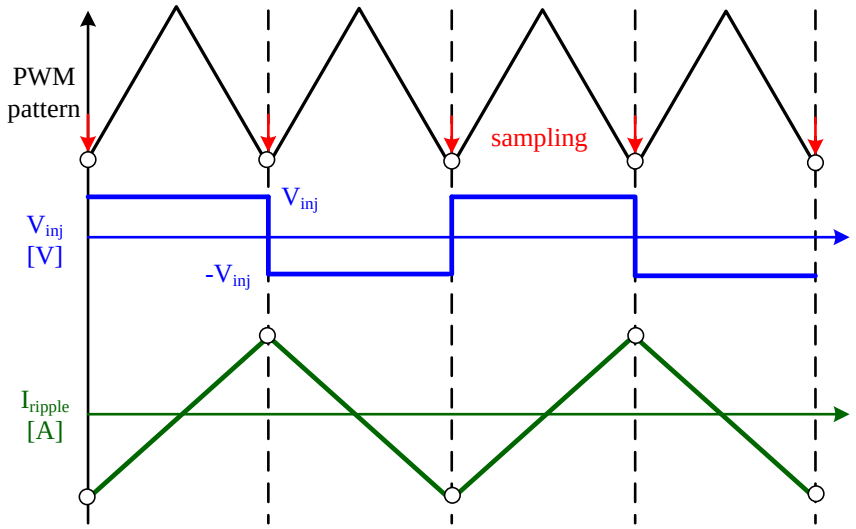


Fig. 3 Square-wave voltage injection with respect to the PWM pattern

Considerations

The magnitude and frequency of the injected signal should be such as to not produce any torque or rotation. Therefore, the chosen frequency should be orders of magnitude greater than the mechanical frequency of the motor.

1) Square-wave type injection signal

Due to the ease of creating a square waveform, injection frequency can be increased up to half of PWM switching frequency (10 KHz over 20 KHz PWM). A high frequency square wave signal, as shown in fig. 3, is superimposed on to the motor voltages in order to create saliency based current ripples in stator current as shown in (1). These current ripples carry position information due to inductance variation.

Let V_{inj} be the injected square wave voltage with an amplitude of V_{squ} such that

$$V_{inj} = \pm V_{squ}$$

If L_S is the saliency-reflected inductance containing the position information, I_{ripple} is the resulting current ripple due to the square wave injection, and ΔT is the sampling time in the microcontroller, then V_{inj} can be expressed as

$$V_{inj} = \pm V_{squ} = L_s(\theta) \times \frac{dI_{ripple}}{dt} = L_s(\theta) \times \frac{\Delta I_{ripple}}{\Delta T} \qquad (1\text{-}1)$$

From this, current ripple can be computed as

$$\Delta I_{ripple} = \pm \Delta T \, L_s^{-1}(\theta) \, V_{squ}$$

Assuming the square-wave voltage is superimposed in the rotor-referred estimated d-axis, as shown in fig.4, then the superimposed voltages in the synchronous frame can be expressed as

$$\begin{bmatrix} v_d^{e'} \\ v_q^{e'} \end{bmatrix} = \begin{bmatrix} \pm v_{inj} \\ 0 \end{bmatrix} \qquad (1\text{-}2)$$

where e' denotes the estimated dq frame and e denotes the real dq frame in fig.4, and θ_{err} is the position error between the estimated and actual reference frames. The inductance matrices in the estimated ($L_{dq}^{e'}$) and actual (L_{dq}^{e}) reference frames can expressed as

$$L_{dq}^{e'} = \begin{bmatrix} \Sigma L - \Delta L\cos(2\theta_{err}) & -\Delta L\sin(2\theta_{err}) \\ -\Delta L\sin(2\theta_{err}) & \Sigma L + \Delta L\cos(2\theta_{err}) \end{bmatrix} \qquad (1\text{-}3)$$

$$L_{dq}^{e} = \begin{bmatrix} \Sigma L - \Delta L & 0 \\ 0 & \Sigma L + \Delta L \end{bmatrix} \qquad (1\text{-}4)$$

where

$\Sigma L = (L_q + L_d)/2$, referred as average inductance

$\Delta L = (L_q - L_d)/2$, referred as differential inductance

The illustration of ΣL and ΔL in $L_q^{e'}$ is shown in fig.5. When θ_{err} becomes zero, $L_{dq}^{e'}$ is totally equal to L_{dq}^{e}.

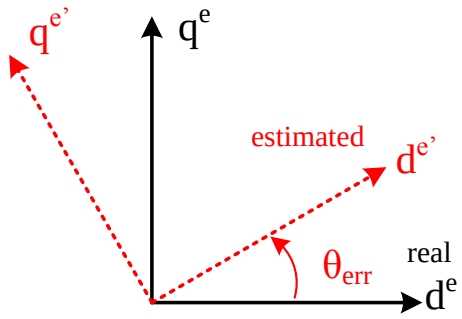


Fig.4 Illustration of estimated and real rotor-referred dq frames

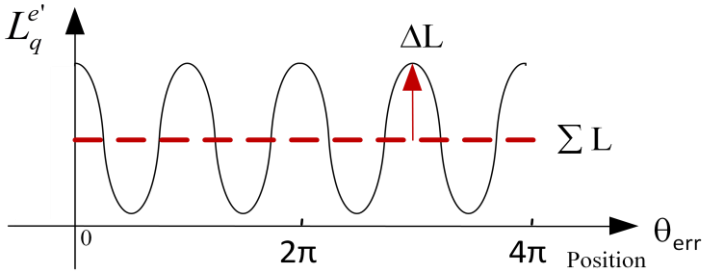


Fig.5 Average inductance ΣL, and differential inductance ΔL, in the inductance waveform

Substituting (1-2) and (1-3) into (1-1), the saliency reflected current ripple, I_{ripple}, can be written as

$$I_{ripple} = \begin{bmatrix} \Delta i_d^{e'} \\ \Delta i_q^{e'} \end{bmatrix} = \Delta T\, L_{dq}^{e'-1} \begin{bmatrix} v_d^{e'} \\ v_q^{e'} \end{bmatrix} = \Delta T \begin{bmatrix} \Sigma L - \Delta L\cos(2\theta_{err}) & -\Delta L\sin(2\theta_{err}) \\ -\Delta L\sin(2\theta_{err}) & \Sigma L + \Delta L\cos(2\theta_{err}) \end{bmatrix}^{-1} \begin{bmatrix} \pm v_{inj} \\ 0 \end{bmatrix}$$

$$= \pm \Delta T\, v_{inj} \begin{bmatrix} \dfrac{\Sigma L + \Delta L(\cos 2\theta_{err})}{\Sigma L^2 - \Delta L^2} \\ \dfrac{\Delta L}{\Sigma L^2 - \Delta L^2}\sin(2\theta_{err}) \end{bmatrix} \approx \pm \begin{bmatrix} I_{\Sigma L} \\ I_{\Delta L}(2\theta_{err}) \end{bmatrix} \text{ (when } \theta_{err} \approx 0) \qquad (1\text{-}5)$$

where,

$$I_{\Sigma L} = \pm \Delta T v_{inj} \frac{\Sigma L + \Delta L}{\Sigma L^2 - \Delta L^2}$$

and, $$I_{\Delta L} = \pm \Delta T v_{inj} \frac{\Delta L}{\Sigma L^2 - \Delta L^2}$$

As mathematically evident, when a high frequency square wave voltage in applied on the estimated d-axis, the current in the estimated q-axis will be proportional to two times θ_{err}. Hence a correction loop can be built using Iq current such that the position of estimated reference frame is continually adjusted until Iq = 0, at which point, the estimated reference frame is aligned with the actual reference frame. A position observer based on θ_{err} is shown in fig.6. Both position and velocity signal are determined by this position observer.

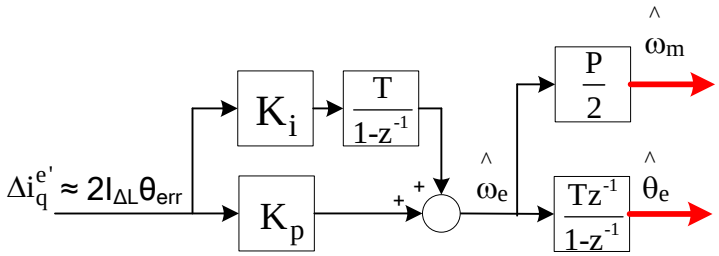


Fig.6 Illustration of position observer in Fig. 1

2) Initial position detection (IPD)

Since Iq current is proportional to twice the angular error, the observer cannot resolve errors above 180deg. Therefore, when the shaft is at rest, the observer algorithm will converge the estimated angle to the plane of the magnet, without guaranteeing to reference North Pole. In other words, the magnetic polarity, north or south, cannot be identified from inductance variation. As a result, additional algorithm is needed to identify the magnetic polarity. The polarity is identified based on different saturation conditions between north and south poles.

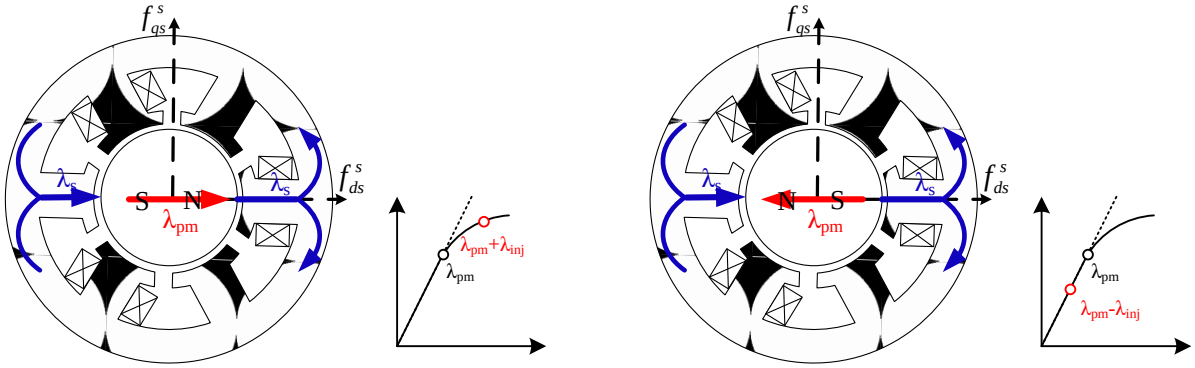


Fig.7 Illustration of the magnetic flux property when the high-frequency signal is injected: (a) the direction of north pole, driving the stator teeth further into saturation, or (b) the direction of south pole, pulling the stator teeth out of saturation

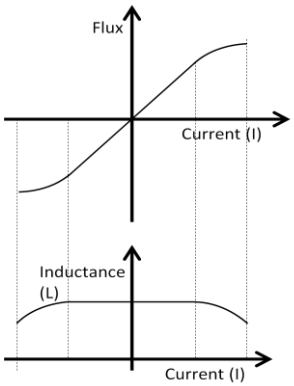
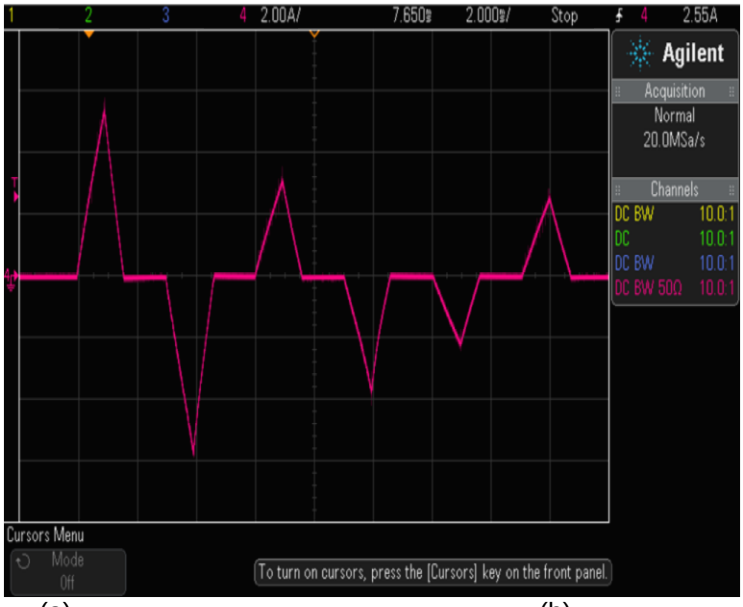


Fig.7. (c) Flux saturation and inductance reduction at higher currents

Fig. 7 illustrates the magnetic flux property when the voltage signal is injected. In (a), when a positive voltage is injected in the direction of north pole, the magnetic flux is augmented. It drives the stator teeth further in saturation, lowering the inductance. In contrast, as shown in (b), when the voltage is injected in the direction of south pole, the flux is reduced. It leads the stator out of saturation, increasing the inductance. Fig.7 c shows the effect of flux and inductance variation as a function of current.

When a short duration voltage pulse is applied on the stator, its transient impedance is more inductive than resistive such that the resistance can be ignored for analysis, and the inductance value will decide the magnitude of the resulting triangular current. Such short duration pulses are applied for each active space vector of inverter, and the vector producing maximum phase current is where the magnetic north pole is located. It is shown in fig.8.



(a) (b)

Fig.8 Inductance reflected high frequency current in the direction of north pole

Algorithm overview

The algorithm for initial position detection is shown in fig.9. When the system is powered on, it starts with a randomly positioned d-axis and gradually converges to the plane of actual d axis, without ensuring the polarity. This completes the primary alignment to $\hat{\theta}_e$ at time T_1. This angle represents the plane of magnet. Until time T_2, no voltage is applied on the motor and the system is put in pause mode to ensure that the motor currents are down to zero. At this point, short duration voltage pulses are applied for each space vector of inverter and the vector producing peak current is identified. This vector has a spread of about 60 degrees, but it will be prudent to consider a span of 120 degrees to accommodate disturbances in current, and angle estimate $\hat{\theta}_e$.If the alignment angle is within this span, then $\hat{\theta}_e$ is assumed to be the location of north pole, if not, it may be implied that it represents the location of south pole. This can be confirmed by adding 180 deg to the alignment angle and cross checking with the angle span for the dominant current vector. If it still fails, which is somewhat unusual, the same set of tests can be repeated for the second dominant current vector.

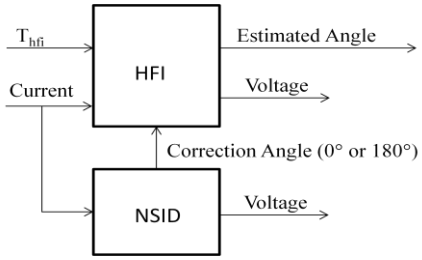


Fig.9 The system block diagram for initial position detection

Implementation

State flow approach is used to implement high frequency injection and initial position detection as shown in fig.10. There are four states, such as RESET state, HFI_IPD state, NSID state and HFI_RUN state.

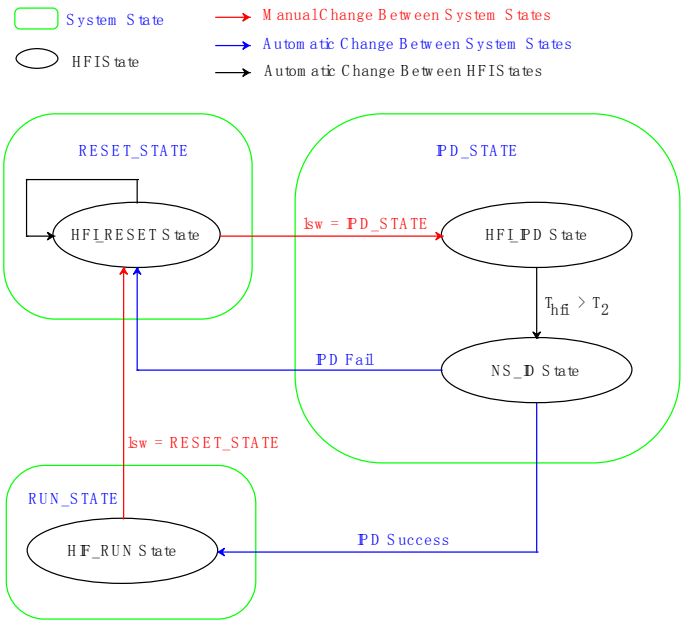


Fig.10. State flow diagram of IPD and HFI

1. RESET state

Initialization of all variables and modules are performed in this state. The next state in the sequence is IPD state, and the transition is initiated by user intervention.

2. HFI_IPD state

In this state, high frequency square wave voltage is imposed on the motor and its currents are measured. As mentioned in the previous sections, the angular position of rotor is identified. The intent of IPD is to align the d axis with respect to N pole of rotor magnet. Since the HFI algorithm ensures the alignment of estimated angle with respect to the plane of magnet, the resulting alignment can be along either N or S pole. Hence it is important to find out the magnet polarity to which the estimated angle is aligned. This is handled by the next state in sequence and the transition is initiated by software itself on a time out basis. The user should select a reasonable time out window to complete the initial angle estimation. Before transition, it is important to ensure that there are no motor currents. This can be achieved by setting the output voltage to zero and setting another time out.

3. NSID state

In this state, high frequency pulses are withdrawn. Instead, a single large pulse is applied for each active switching state of the inverter to drive more current into the motor such that the switching state that is close in alignment to the magnet will saturate the flux and draw more current than other switching states. Giving a margin of 60° on either side of this vector, the rotor should lie within this 120° arc. If the estimated angle lies within this arc, then it can be construed that the estimated angle corresponds to N pole. If this test fails, the estimated angle is rotated by 180° to verify if the new angle is within arc. If it succeeds, then estimated angle corresponds to S pole. If this test also fails, repeat the same sequence using the second most dominant switching state. If this also fails, then it transitions the control to RESET state, else to RUN state.

4. HFI_RUN state

High frequency injection is restarted in this state again, and the magnitude is chosen based on load conditions and transient requirements. Higher magnitude helps to identify the angle better because of more signal imprint, but will lead to more distortion in phase current. Hence a compromise choice should be made based on need.

IMPORTANT NOTICE

Texas Instruments Incorporated and its subsidiaries (TI) reserve the right to make corrections, modifications, enhancements, improvements, and other changes to its products and services at any time and to discontinue any product or service without notice. Customers should obtain the latest relevant information before placing orders and should verify that such information is current and complete. All products are sold subject to TI's terms and conditions of sale supplied at the time of order acknowledgment.

TI warrants performance of its hardware products to the specifications applicable at the time of sale in accordance with TI's standard warranty. Testing and other quality control techniques are used to the extent TI deems necessary to support this warranty. Except where mandated by government requirements, testing of all parameters of each product is not necessarily performed.

TI assumes no liability for applications assistance or customer product design. Customers are responsible for their products and applications using TI components. To minimize the risks associated with customer products and applications, customers should provide adequate design and operating safeguards.

TI does not warrant or represent that any license, either express or implied, is granted under any TI patent right, copyright, mask work right, or other TI intellectual property right relating to any combination, machine, or process in which TI products or services are used. Information published by TI regarding third-party products or services does not constitute a license from TI to use such products or services or a warranty or endorsement thereof. Use of such information may require a license from a third party under the patents or other intellectual property of the third party, or a license from TI under the patents or other intellectual property of TI.

Reproduction of TI information in TI data books or data sheets is permissible only if reproduction is without alteration and is accompanied by all associated warranties, conditions, limitations, and notices. Reproduction of this information with alteration is an unfair and deceptive business practice. TI is not responsible or liable for such altered documentation. Information of third parties may be subject to additional restrictions.

Resale of TI products or services with statements different from or beyond the parameters stated by TI for that product or service voids all express and any implied warranties for the associated TI product or service and is an unfair and deceptive business practice. TI is not responsible or liable for any such statements.

TI products are not authorized for use in safety-critical applications (such as life support) where a failure of the TI product would reasonably be expected to cause severe personal injury or death, unless officers of the parties have executed an agreement specifically governing such use. Buyers represent that they have all necessary expertise in the safety and regulatory ramifications of their applications, and acknowledge and agree that they are solely responsible for all legal, regulatory and safety-related requirements concerning their products and any use of TI products in such safety-critical applications, notwithstanding any applications-related information or support that may be provided by TI. Further, Buyers must fully indemnify TI and its representatives against any damages arising out of the use of TI products in such safety-critical applications.

TI products are neither designed nor intended for use in military/aerospace applications or environments unless the TI products are specifically designated by TI as military-grade or "enhanced plastic." Only products designated by TI as military-grade meet military specifications. Buyers acknowledge and agree that any such use of TI products which TI has not designated as military-grade is solely at the Buyer's risk, and that they are solely responsible for compliance with all legal and regulatory requirements in connection with such use.

TI products are neither designed nor intended for use in automotive applications or environments unless the specific TI products are designated by TI as compliant with ISO/TS 16949 requirements. Buyers acknowledge and agree that, if they use any non-designated products in automotive applications, TI will not be responsible for any failure to meet such requirements.